ARUW Vision Technical Question



IMPORTANT!

Please read the task specification in detail to ensure you cover everything. We look for purposeful and thorough responses. Read the ENTIRE document, as it has been written for you to learn from and to respect your time.

You will design and implement the decision making for an autonomous agent in a game. This task was chosen to demonstrate what work in Vision looks like.

In this booklet: **1** State machines · **2** The game · **3** Controller inputs · **4** The task · **5** Resources and FAQ

The task

Design and implement a **state-machine-based controller** for a game agent that plays a territory-capture game (see Section 4).

What you submit

- Your controller code
- A concise state-machine design

How we review

Your state-machine design and reasoning, code implementation, extensibility, and testability. Game score is useful feedback.

Ground rules

- This question is open-ended and supports many approaches. **There is no one correct answer.**
- Spend as much or as little time as you like. More time does not necessarily produce a stronger solution.
- No prior game-AI or robotics experience is assumed. This booklet and the starter project provide the background needed.
- Starter projects are provided in **Java, Python, and C++**. Choose the language you are most comfortable with.

AI tools

AI tools may be used for brainstorming, learning, design, debugging, and coding.

Please understand every design choice and every part of the code you submit. We will ask you to explain your thought process and implementation during the interview.

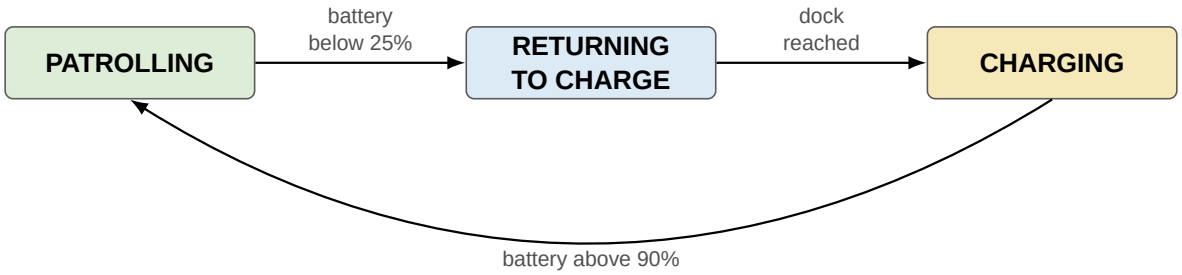
1 State Machines

Your game agent will have different goals at different moments. A state machine gives you a structured way to organize those behaviors and decide when the agent should switch between them.

Controller	the decision-making code.
State	the behavior the controller is currently running.
Transition	the condition that moves the controller into another state.
State machine	the states and their transition rules together.

1.1 A Simple Example

Imagine an autonomous rover that patrols an area, returns to its charging dock when the battery becomes low, and waits at the dock while charging. Here, the controller is the rover's decision-making code.



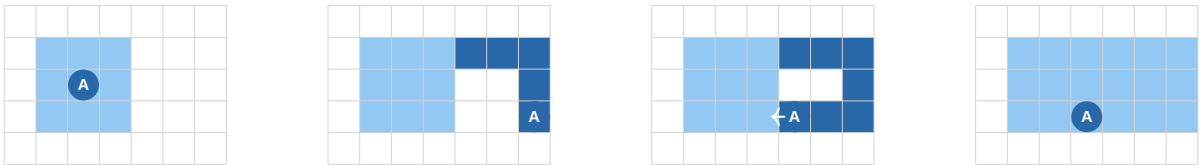
Each state can contain its own decision logic. For this question, you will use the same idea to organize the behavior of a game agent. You decide which states are useful, what each state does, and when transitions should happen.

2 The Game

This game may be familiar to you from *Paper.io*. If not, the rules below are everything you need for this task.

50 × 50	2000	3 × 3	11 × 11
board	turns per player	starting territory	visible window

Two agents play on a **50 x 50 grid**. Each player begins inside a small area of owned territory. Moving outside owned territory creates an exposed trail. Returning to territory you own closes that trail and captures the enclosed area.



1. Start inside owned territory — **2. Leave and create a trail** — **3. Reconnect to owned territory** — **4. Trail and enclosed cells become yours**

Light blue shows owned territory. Dark blue shows the active trail.

2.1 Rules

Movement

- Move one cell north, south, east, or west.
- Each player receives **2000 turns**.
- A successful move consumes one turn.

Invalid move: if your agent moves off the board or into the opponent's current cell, the game harness calls your controller for another move on the same turn.

Territory and capture

- Each player starts with a **3 x 3** territory.
- Leaving owned territory creates an active trail.
- Re-entering owned territory captures the trail and enclosed cells.
- Enclosed opponent territory changes ownership.

Trails and respawning

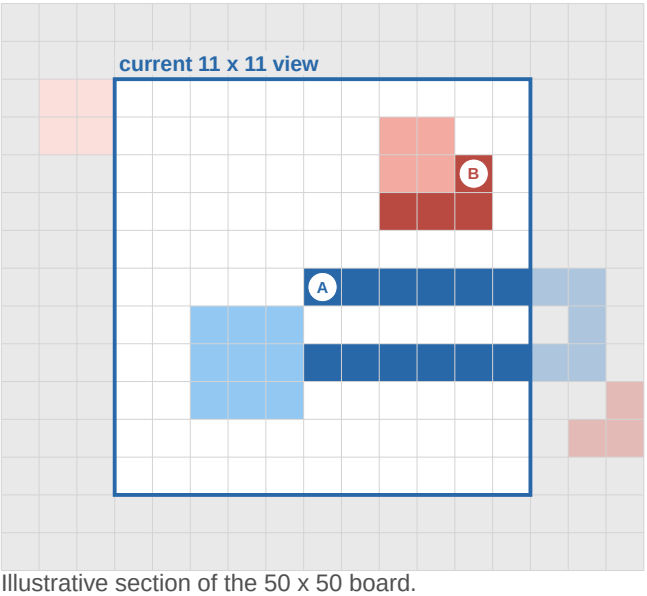
- Stepping on your own trail kills your agent.
- Stepping on the opponent's trail kills their agent and completes your move.
- Death clears that player's active trail and current territory.
- Their starting territory is restored and the agent respawns.

Winning the match

- The match ends after both players use all turns.
- Current territory count determines the winner.
- Equal territory counts produce a draw.

3 What Your Controller Works With

3.1 What Your Agent Can See



Illustrative section of the 50 x 50 board.

Your controller receives an **11 x 11 visible grid** centered on the agent. Near a board edge, this window is clipped to the board. The figure shows a larger section of the game so you can see the difference between the current observation and the rest of the board.

Cells outside the outlined window are shown faded only to explain the visibility boundary. Your visible grid contains the free cells, territory, trails, and agents inside the window.

Your **complete active trail** is supplied separately, so its full path remains available even when part of it extends outside the window.

- your territory
- your trail
- opponent territory
- opponent trail
- outside current view

3.2 How the Framework Works

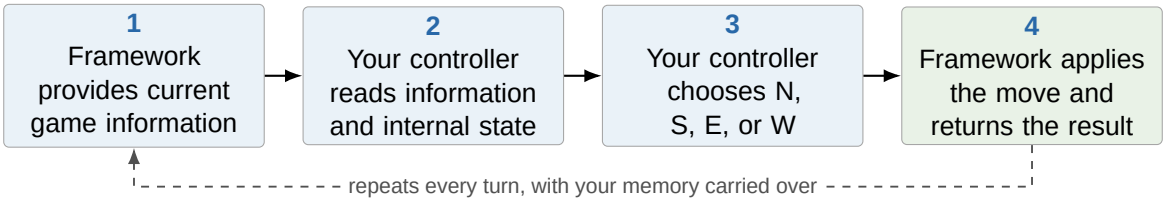
The framework runs the game; your job is the **controller**, which reads the current turn's information and chooses one movement direction.

Framework handles

- authoritative board and game state
- movement validation and game rules
- capture, death, and respawning
- turns, score, and GUI

Your controller handles

- current state and transitions
- strategy, memory, and derived information
- choosing one movement direction
- updating state after each result



On the next call, your controller can use the state and memory it kept from previous turns together with the new information from the framework.

3.3 Information Available to Your Controller

Exact method names and language-specific types are documented in the candidate guide for your chosen starter project. Conceptually, your controller receives the following information.

Your agent

- current position
- respawn position
- complete active trail
- remaining turns

The match

- your territory count
- opponent territory count
- full board width and height

Current observation

- the current visible grid
- visible free cells
- visible territory and trails
- visible agents

After a move

- whether the move completed normally
- whether it captured territory
- whether your agent died
- whether the move was invalid

Information you can derive and remember

The values above are the inputs supplied by the framework. Your controller can compute and remember additional information that helps its strategy. For example, it could calculate distances, remember previously seen cells, track where an opponent or trail was last observed, estimate how far an exposed trail is from safety, compare territory counts over time, or build its own internal map. What you derive and remember is part of your design.

4 The Task

Design and implement a **state-machine-based controller** for your agent.

1 Design your state machine

- Choose the states your controller uses.
- Define what each state is responsible for.
- Define the important transitions between states.
- Decide what information or memory each behavior needs.
- Be ready to explain why you made these design choices.

2 Implement your controller

- Implement your own controller in the provided candidate file.
- Use the information supplied by the framework to choose one movement direction each turn.
- Keep any internal state, memory, or supporting classes that make sense for your design.
- Use, adapt, or replace the provided helpers as useful for your approach.

3 Submit

- **Your controller code**, including any supporting classes you created.
- **A concise state-machine design** showing the states, the purpose of each state, the important transitions, and your thought process. A diagram, written description, or both are suitable.

Things to think about

- What should each state be responsible for?
- What observations should cause a transition?
- How long is it worth leaving a trail exposed?
- When should the agent return to safety?
- When is an exposed opponent trail worth pursuing for a kill?
- What information is useful to remember?
- How should territory count and remaining turns change behavior?

5 Resources and FAQ

5.1 Starter Projects

Separate starter projects are provided for **Java, Python, and C++**. Each contains the same game, GUI, candidate controller area, examples, helpers, setup instructions, and language-specific candidate guide.

Java	gitlab.com/aruw/vision/applications/java-state-machine-starter
Python	gitlab.com/aruw/vision/applications/python-state-machine-starter
C++	gitlab.com/aruw/vision/applications/cpp-state-machine-starter

5.2 Provided Helpers

The starter projects include helpers for common board calculations and memory: next positions, bounds and mechanically invalid destinations, distances, searching the visible grid, and remembering earlier observations. Use them as provided, adapt them, or build your own equivalents.

5.3 Example Controllers

BasicStateMachine

A minimal two-state reference example:

- `EXPANDING`: chooses a mechanically valid direction at random and grows an active trail.
- `RETURNING`: starts when the active trail reaches 4 cells and heads roughly toward the respawn point while avoiding its own trail.

A closed trail, capture, or death resets the example to `EXPANDING`. Read it to see how state, transitions, and move results fit together, then build your own controller in the candidate file.

EnemyStateMachine

The GUI can run your controller against `EnemyStateMachine` (**Enemy State Machine**), a stronger state-machine opponent developed as a reference solution.

- Select it from the GUI's opponent list to play or watch against it.
- Use it as a benchmark once your basic states are working.
- Its source is available if you want to see how the framework's move calls are used in a complete controller.

You may inspect its implementation to understand the framework, but your submission should reflect your own states, transitions, and reasoning.

5.4 How to Get Started

1. Choose a language above and follow its setup instructions.
2. Run the GUI and watch an example match.
3. Read the candidate guide and the basic example controller.
4. Sketch your own states, transitions, and useful memory.
5. Implement your controller in the provided candidate file.
6. Play against the provided opponents, including `EnemyStateMachine`, and refine your design.

5.5 FAQ

I am a bit overwhelmed. Where should I begin?

Start with a small state machine that completes turns reliably. Get one or two behaviors working, watch them in the GUI, and add complexity when you have a reason for it.

How do I set up an AI assistant?

If you don't have access to any coding tools, please use this guide for a free, easy setup: https://gitlab.com/aruw/vision/applications/ai-assistant-setup-guide/-/blob/main/ai-assistant-setup-guide.pdf.

Questions after reading this booklet and your language-specific candidate guide are welcome at robomstr@uw.edu.